ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
-------*-------**

BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI:

Tái hiện và cải tiến Smart HPA: auto-scaling tiết kiệm tài nguyên cho kiến trúc microservice trên cluster k3s

Giảng viên hướng dẫn: **[Tên giảng viên]**
Nhóm sinh viên thực hiện: **Hoàng Quốc Dương (23020593)**
[Thành viên 2 – MSSV]
[Thành viên 3 – MSSV]
Lớp học phần: **[Mã lớp học phần]**

HÀ NỘI, 2026

MỤC LỤC

1. Giới thiệu và mục tiêu 6
1.1. Mục tiêu 6
1.2. Phạm vi 6
1.3. Tóm tắt kết quả 6
2. Cơ sở lý thuyết 8
2.1. Auto-scaling ngang và Kubernetes HPA 8
2.2. Vòng điều khiển MAPE-K 8
2.3. Các loại chính sách scaling 8
2.4. Kiến trúc điều khiển 8
2.5. Mô hình dự báo PROPHET 9
3. Tóm tắt bài báo 10
3.1. Smart HPA 10
3.2. ProSmart HPA 10
3.3. Chỉ số đánh giá 10
3.4. Thiết lập và kết quả chính của bài báo 11
4. Môi trường và phương pháp thực nghiệm 12
4.1. Hạ tầng 12
4.2. Ứng dụng thử nghiệm 12
4.3. Cài đặt các auto-scaler 12
4.4. Thiết kế thí nghiệm 13
4.5. Quy trình chạy và kiểm tra hợp lệ 13
4.6. Khác biệt so với bài báo 13
5. Kết quả tái hiện 15
5.1. Kịch bản 5R-50% 15
5.2. Kịch bản 2R-20% 18
5.3. ProSmart HPA (2R-20%, window 25 giây) 20
5.4. Tổng hợp tính nhất quán 22
6. Phân tích điểm yếu của bài báo 24
7. Đề xuất cải tiến và kết quả 25
7.1. C1: Đo chỉ số phía người dùng 25
7.2. C3: Chia quota theo độ ưu tiên 26
7.3. C2: Tải đột biến 28
8. Thảo luận và hạn chế 33
8.1. Mức độ tái hiện 33
8.2. Hạn chế của bài tập 33

9. Kết luận và hướng phát triển 34
Tài liệu tham khảo 35
Phụ lục A. Mã nguồn và dữ liệu 36

Danh mục từ viết tắt

Viết tắt	Ý nghĩa
HPA	Horizontal Pod Autoscaler: bộ tự động co giãn ngang mặc định của Kubernetes
K8s HPA	HPA mặc định của Kubernetes, dùng làm baseline
MAPE-K	Monitor – Analyze – Plan – Execute – Knowledge: vòng điều khiển tự thích nghi
ARM	Application Resource Manager: tầng quản lý tài nguyên mức ứng dụng của Smart HPA
CR / DR / maxR	Số replica hiện tại / số replica mong muốn / số replica tối đa (quota) của một service
CMV / TMV	Giá trị metric hiện tại / giá trị metric mục tiêu (ngưỡng CPU)
5R-50%	Kịch bản maxR = 5 replica, ngưỡng CPU 50% (tương tự cho 2R-20%)
m (milliCPU)	Một phần nghìn nhân CPU; 1000m = 1 vCPU
p95	Phân vị thứ 95 của thời gian phản hồi
SLO	Service Level Objective: mục tiêu chất lượng dịch vụ
TB ± ĐLC	Trung bình ± độ lệch chuẩn

Lời cam đoan và cảm ơn

Nhóm em xin cam đoan rằng trong quá trình thực hiện và hoàn thiện báo cáo này, nhóm có sử dụng các công cụ trí tuệ nhân tạo (như Claude) để hỗ trợ lập trình script thí nghiệm, tổng hợp số liệu và chỉnh sửa diễn đạt.

Toàn bộ thí nghiệm trong báo cáo do nhóm tự dựng hạ tầng và tự chạy trên cluster của nhóm. Mọi số liệu, bảng và biểu đồ đều sinh ra từ dữ liệu thô lưu trong repo của nhóm và có thể kiểm tra lại. Các lần chạy đi ngược xu hướng được giữ lại và nêu rõ, không bị loại bỏ.

Nhóm em xin gửi lời cảm ơn chân thành tới **[Tên giảng viên]** vì sự định hướng, góp ý và hỗ trợ trong suốt quá trình thực hiện bài tập lớn.

Nhóm em xin chịu hoàn toàn trách nhiệm về tính trung thực, minh bạch và liêm chính học thuật của toàn bộ nội dung trong báo cáo này.

Đại diện nhóm,
Hoàng Quốc Dương

1. Giới thiệu và mục tiêu

Kiến trúc microservice chia một ứng dụng thành nhiều dịch vụ nhỏ, mỗi dịch vụ được triển khai và mở rộng độc lập. Trên Kubernetes, cơ chế mở rộng mặc định là **Horizontal Pod Autoscaler (HPA)**: mỗi deployment có một HPA riêng, tăng hoặc giảm số replica dựa trên mức sử dụng CPU so với một ngưỡng, trong khoảng [minR, maxR] cố định.

Cách làm này có hai hạn chế. Thứ nhất, mỗi service bị chặn bởi maxR của riêng nó: khi tải cao, service bận bị kẹt ở maxR và thiếu tài nguyên, trong khi service rảnh vẫn giữ phần quota không dùng đến. Thứ hai, pod cần vài chục giây để khởi động hoặc tắt nên HPA luôn phản ứng chậm hơn tải.

Ahmad và cộng sự [1] đề xuất hai bộ auto-scaler để giải quyết hai vấn đề trên: **Smart HPA** (phản ứng, chia sẻ quota maxR giữa các service) và **ProSmart HPA** (chủ động, dự đoán nhu cầu bằng PROPHET). Bài báo đánh giá trên ứng dụng Online Boutique chạy trên Amazon EKS và báo cáo Smart HPA tốt hơn K8s HPA ở mọi kịch bản.

1.1. Mục tiêu

1. **Tái hiện** thí nghiệm so sánh Smart HPA với K8s HPA ở quy mô nhỏ (cluster k3s 3 node trên một laptop), ở hai kịch bản 5R-50% và 2R-20%, và ProSmart HPA ở 2R-20% với prediction window 25 giây. Mục tiêu là thấy lại **xu hướng** của bài báo, không cần trùng con số.
2. **Kiểm tra tính nhất quán** của xu hướng ở **từng lần chạy** chứ không chỉ ở giá trị trung bình (yêu cầu của giảng viên). Mỗi cấu hình chạy 5 lần, báo cáo trung bình ± độ lệch chuẩn, số lần đúng hướng và kiểm định Mann-Whitney U.
3. **Phân tích điểm yếu** của bài báo, đề xuất và thử nghiệm cải tiến: C1 (chỉ số phía người dùng), C2 (tải đột biến) và C3 (chia quota theo độ ưu tiên).

1.2. Phạm vi

Trong phạm vi: Smart HPA, K8s HPA, ProSmart HPA (chỉ window 25 giây), hai kịch bản 5R-50% (kịch bản chính) và 2R-20% (kịch bản khó nhất của bài báo). Ngoài phạm vi: đủ 9 kịch bản × 10 lần chạy và 6 prediction window như bài báo (khoảng 60 giờ chạy, vượt quá thời gian của bài tập).

1.3. Tóm tắt kết quả

- **5R-50%:** Smart HPA tốt hơn K8s HPA ở 6/7 chỉ số của bài báo trong **5/5 lần chạy**; chỉ số còn lại (Overutilization Time) đúng hướng 4/5 lần nhưng gần như ngang nhau. Khớp xu hướng bài báo.

- **2R-20%:** không "gần ngang nhau" như bài báo. Smart HPA tốt hơn ở Overprovision và p95 latency (5/5) nhưng kém hơn ở CPU Underprovision và CPU Overutilization (5/5). Hướng chênh lệch vẫn lặp lại ở mọi lần chạy.
- **ProSmart HPA:** giảm Overutilization 38,6% và p95 latency 25% so với Smart HPA (5/5), nhưng dùng **vượt tổng quota 16–23%** do một nhánh của Algorithm 4 (điểm yếu W8).
- **Phát hiện W9:** code ARM trong replication package làm **mất 25–37% tổng quota** ở 5R-50%. Chỉ cần sửa lỗi này, CPU Underprovision giảm từ 389m xuống 10m, khớp con số 0m của bài báo.
- **Cải tiến C3** (ưu tiên theo vai trò service) chỉ giảm thêm Overutilization ở 5R-50% (87% → 75%, p = 0,036), không cải thiện ở 2R-20%.
- **Cải tiến C2** (tải đột biến, 5R-50%, 3 cặp): Smart HPA vẫn tốt hơn K8s HPA ở 6/7 chỉ số tài nguyên (3/3 lần) nhưng hòa về phía người dùng và bị flapping: khoảng 43 lần đảo chiều scale mỗi lần chạy, K8s HPA 0 lần.

2. Cơ sở lý thuyết

2.1. Auto-scaling ngang và Kubernetes HPA

Auto-scaling ngang (horizontal scaling) thay đổi **số bản sao** (replica) của một service, khác với scaling dọc (vertical) thay đổi CPU/RAM cấp cho từng bản sao. Kubernetes HPA đọc mức sử dụng CPU của các pod qua metrics-server, rồi tính số replica mong muốn:

$$DR = \lceil CR \times CMV / TMV \rceil$$

trong đó CR là số replica hiện tại, CMV là mức CPU hiện tại (phần trăm so với CPU request) và TMV là ngưỡng mục tiêu. DR sau đó bị kẹp trong [minR, maxR]. Khi scale down, HPA dùng cửa sổ ổn định (stabilization window) mặc định 300 giây để tránh dao động.

2.2. Vòng điều khiển MAPE-K

MAPE-K là mô hình tham chiếu cho hệ thống tự thích nghi, gồm bốn pha **Monitor** (thu thập metric), **Analyze** (so với mục tiêu), **Plan** (quyết định hành động), **Execute** (thực thi) và một **Knowledge Base** dùng chung để lưu trạng thái, lịch sử. Cả Smart HPA và ProSmart HPA đều được thiết kế theo vòng này; ProSmart thêm thành phần Intelligence thành MAPE-KI.

2.3. Các loại chính sách scaling

Loại	Ý tưởng	Ví dụ
Dựa trên ngưỡng	So metric với ngưỡng cố định, vượt thì thêm/bớt replica	K8s HPA, Smart HPA
Logic mờ	Luật IF–THEN với biến mờ thay cho ngưỡng cứng	Fuzzy controller
Lý thuyết hàng đợi	Mô hình hóa service như hàng đợi để tính số máy cần thiết	M/M/c
Lý thuyết điều khiển	Bộ điều khiển phản hồi giữ metric quanh điểm đặt	PID
Học máy / dự báo	Dự báo tải hoặc nhu cầu tài nguyên rồi scale trước	PROPHET, LSTM, RL

2.4. Kiến trúc điều khiển

- **Tập trung:** một bộ điều khiển quyết định cho mọi service. Có cái nhìn toàn cục nhưng là điểm nghẽn và điểm lỗi duy nhất.
- **Phi tập trung:** mỗi service tự quyết (như K8s HPA). Đơn giản, mở rộng tốt nhưng không phối hợp được tài nguyên giữa các service.
- **Phân cấp:** tầng dưới phi tập trung xử lý trường hợp thường, tầng trên tập trung chỉ bật khi cần phối hợp. Smart HPA thuộc loại này.

2.5. Mô hình dự báo PROPHET

PROPHET là mô hình dự báo chuỗi thời gian dạng cộng: $y(t) = g(t) + s(t) + h(t) + \Sigma\beta \cdot x(t) + \varepsilon$, gồm xu hướng g(t), mùa vụ s(t), ngày lễ h(t) và các biến hồi quy bổ sung x(t). ProSmart HPA dùng PROPHET với các biến hồi quy là CPU hiện tại, ngưỡng CPU và số replica hiện tại để dự đoán số replica cần sau một khoảng thời gian (prediction window).

3. Tóm tắt bài báo

Bài báo [1] (Journal of Systems and Software, 2025) là bản mở rộng của bài Smart HPA tại ICSA 2024 [2]. Đóng góp mới của bản 2025 là ProSmart HPA. Baseline của toàn bộ bài là Kubernetes HPA.

3.1. Smart HPA

Smart HPA có kiến trúc phân cấp hai tầng dựa trên MAPE-K:

Thành phần	Vai trò
Microservice Manager (mỗi service một cái, phi tập trung)	Algorithm 1: tính DR = ⌈CR × CMV/TMV⌉, quyết định scale up/down như K8s HPA
Microservice Capacity Analyzer	Kiểm tra DR ≤ maxR cho mọi service; nếu có service DR > maxR thì gọi tầng trên
Application Resource Manager (tập trung, chỉ bật khi thiếu)	Algorithm 2: tìm service thừa và thiếu, chuyển quota maxR từ service thừa sang service thiếu, ưu tiên service thiếu nhiều nhất

Điểm mấu chốt: **tổng quota của cả ứng dụng không đổi**, Smart HPA chỉ phân phối lại quota giữa các service. Nhờ đó service bận có thể vượt maxR ban đầu của nó mà không cần thêm tài nguyên cho cả ứng dụng.

3.2. ProSmart HPA

- Mở rộng MAPE-K thành MAPE-KI, thêm thành phần Intelligence dùng 22 model PROPHET (11 dự đoán nhu cầu, 11 dự đoán capacity), train trên Knowledge Base của các lần chạy Smart HPA.
- **Algorithm 3** (Microservice Manager): PDR = PROPHET(metric, ngưỡng, CR); so PDR với CR để ra quyết định; nếu PDR > maxR thì gọi ARM.
- **Algorithm 4** (ARM): dự đoán PmaxR cho cả 11 service, cân bằng phần quota thêm và bớt để tổng không đổi, rồi ra ResDR, ResSD cho từng service.
- Prediction window tốt nhất là **25 giây**, nằm giữa thời gian khởi động pod (15 giây) và thời gian tắt pod (30 giây).

3.3. Chỉ số đánh giá

Bảng 3.1: Bảy chỉ số đánh giá (Table 1 của bài báo)

Chỉ số	Ý nghĩa	Tốt khi
Supply CPU (m)	CPU của các replica đang được cấp	Cao
CPU Overutilization (%)	CPU % vượt ngưỡng (dưới ngưỡng tính 0), cộng trên 11 service	Thấp
Overutilization Time (phút)	Thời gian có ít nhất một service vượt ngưỡng	Thấp

Chỉ số	Ý nghĩa	Tốt khi
CPU Overprovision (m)	CPU thừa = capacity − demand	Thấp
Overprovision Time (phút)	Thời gian không service nào thiếu CPU	Cao
CPU Underprovision (m)	CPU thiếu = demand − capacity	Thấp
Underprovision Time (phút)	Thời gian có ít nhất một service thiếu CPU	Thấp

3.4. Thiết lập và kết quả chính của bài báo

Bài báo chạy trên 10 node AWS EC2 t3.medium (Amazon EKS, Kubernetes 1.24), tải Locust 600 users tăng dần trong 5 phút, tổng 15 phút; 9 kịch bản (maxR 2/5/10 × ngưỡng 20/50/80%), mỗi kịch bản 10 lần.

Bảng 3.2: Kết quả chính của bài báo cần đối chiếu

Kịch bản	Kết quả (Smart HPA so với K8s HPA)
5R-50% (tốt nhất)	Underprovision 0 so với 934,04m (13,46 phút); Overutilization 19,30% so với 98,06% (giảm 5,08 lần); Overutilization Time giảm 1,98 lần
2R-20% (khó nhất)	Gần ngang nhau, cải thiện khoảng 1,004–1,01 lần
10R-20%	Supply CPU cao hơn 1,83 lần (11.188,76m so với 6.110,41m)
ProSmart, 2R-20%, window 25s	So với Smart HPA: Overutilization −25,40%, Underprovision −25,34%, Overprovision −28,24%, Supply +2,17%

4. Môi trường và phương pháp thực nghiệm

4.1. Hạ tầng

Toàn bộ thí nghiệm chạy trên một laptop Dell Precision 7670 (Intel i9-12950HX 16 nhân/24 luồng, 32GB RAM, Windows) với VMware Workstation Pro. Cluster gồm 3 máy ảo Ubuntu Server 24.04.5 nối qua mạng NAT VMnet8 (192.168.161.0/24), cài k3s `v1.36.5+k3s1` (bản phân phối Kubernetes đạt chứng nhận CNCF) với `--disable traefik` để giải phóng cổng 80 cho frontend.

Bảng 4.1: Cấu hình cluster

Máy ảo	IP	vCPU	RAM	Vai trò
k3s-master	192.168.161.10	4	4GB	Control plane + chạy pod
k3s-worker1	192.168.161.11	4	4GB	Worker
k3s-worker2	192.168.161.12	4	4GB	Worker

Các auto-scaler, script ghi metric và Locust chạy trên chính máy host (Python 3.11), điều khiển cluster bằng `kubectl` qua mạng nội bộ.

4.2. Ứng dụng thử nghiệm

Ứng dụng là **Online Boutique** của Google (11 microservice, image `v0.8.1`), triển khai từ manifest trong replication package của bài báo. Resource request/limit giữ nguyên như bài báo:

Bảng 4.2: CPU request/limit của các service

Service	CPU request / limit
emailservice, checkoutservice, frontend, paymentservice, productcatalogservice, currencyservice, shippingservice	100m / 200m
recommendationservice	100m / 200m (RAM 220Mi)
cartservice, adservice	200m / 300m
redis-cart	70m / 125m

4.3. Cài đặt các auto-scaler

- **K8s HPA:** `kubectl autoscale <deploy> --cpu=50% --min=1 --max=5` cho cả 11 deployment (2R-20%: `--cpu=20% --max=2`). Vì K8s HPA không tự ghi log, nhóm viết một bộ quan sát dùng lại đúng pha Monitor/Analyse của Smart HPA để ghi Knowledge Base cùng định dạng, trong đó cột DR là giá trị **chưa bị chặn bởi maxR** (nếu dùng số replica mong muốn mà HPA báo, vốn đã bị chặn bởi maxR, thì Underprovision luôn bằng 0).
- **Smart HPA:** dùng code trong replication package, sửa tối thiểu để chạy được: tên file import có dấu cách, cấu trúc thư mục, gom maxR/ngưỡng từ 22 dòng

hard-code về một file cấu hình, thêm retry 5 lần cho `kubectl`. **Thuật toán giữ nguyên**, kể cả lỗi ở dòng 101 của ARM (lệch Algorithm 2).

- **ProSmart HPA:** code trong repo không chạy được (ít nhất 7 lỗi), nên nhóm viết lại controller theo Algorithm 3–4. 22 model PROPHET đi kèm được tác giả train từ Knowledge Base các lần chạy Smart HPA trên cluster của họ, và chỉ dùng thời gian làm đầu vào, không có biến hồi quy như bài báo mô tả (W7). Nhóm làm lại bước train này trên cluster của nhóm: train 22 model từ Knowledge Base 3 lần chạy Smart HPA 2R-20% của nhóm, có biến hồi quy theo mục 5.1 của bài. Bằng chứng cho việc train lại ở mục 5.3 (Bảng 5.4, Hình 5.6).

4.4. Thiết kế thí nghiệm

Bảng 4.3: Các biến của thí nghiệm

Loại	Biến	Giá trị
Độc lập	Auto-scaler	K8s HPA, Smart HPA; ProSmart HPA (chỉ 2R-20%)
Độc lập	Kịch bản (maxR, ngưỡng)	5R-50%, 2R-20%
Cố định	Tải	Locust, 200 users, ramp 5 phút (0,67 user/s), tổng 15 phút
Cố định	Ứng dụng, cluster	Online Boutique mặc định, 3 node như mục 4.1
Phụ thuộc	Chỉ số	7 chỉ số ở Bảng 3.1 + chỉ số phía người dùng (C1)

Số user giảm từ 600 xuống 200 cho phù hợp cluster 12 vCPU, nhưng giữ thời gian ramp 5 phút như bài báo. Chạy thử với 100 users trong 3 phút thì K8s HPA không chạm maxR nên không phân biệt được hai auto-scaler; 200 users đủ để `frontend` và `currencyservice` của K8s HPA bị kẹt ở maxR.

4.5. Quy trình chạy và kiểm tra hợp lệ

Mỗi lần chạy được tự động hóa bằng script: reset (xóa HPA, đưa mọi deployment về 1 replica) → nghỉ 300 giây cho CPU nguội → bật auto-scaler, bộ ghi trạng thái cluster và Locust trong 15 phút → dọn dẹp → kiểm tra → sinh báo cáo. Các lần chạy được xếp **xen kẽ** K8s HPA và Smart HPA theo cặp để giảm sai lệch theo thời gian.

Bước kiểm tra tự động loại lần chạy bị mất kết nối (cửa sổ 10 giây gần như không có request, lỗi > 1%, hoặc mất > 1 phút số liệu). Về sau, nhóm ghi thêm ping mỗi giây tới cả 3 node để xác định sự cố mạng. Một lần chạy (K8s HPA 5R-50% run 4) bị loại do 3 lần mất kết nối và đã chạy lại. Không lần chạy hợp lệ nào có pod `Unschedulable`.

4.6. Khác biệt so với bài báo

Bảng 4.4: Khác biệt giữa thí nghiệm của nhóm và bài báo

Yếu tố	Bài báo	Nhóm
Hạ tầng	10 node AWS EC2 t3.medium	3 VM VMware trên 1 laptop

Yếu tố	Bài báo	Nhóm
Kubernetes	Amazon EKS, K8s 1.24	k3s v1.36.5 (K8s 1.36)
Nơi chạy auto-scaler, Locust	Máy local, kết nối AWS qua internet	Máy host, mạng nội bộ
Tải	600 users, ramp 5 phút	200 users, ramp 5 phút
Kịch bản	9	2 (5R-50%, 2R-20%)
Số lần chạy	10 lần/kịch bản	5 lần/kịch bản
ProSmart HPA	6 window, model train từ 10 lần chạy Smart HPA của tác giả	Window 25s, model train lại từ 3 lần chạy Smart HPA của nhóm, cùng quy trình

5. Kết quả tái hiện

Tổng cộng 20 lần chạy hợp lệ cho phần tái hiện (2 kịch bản × 2 auto-scaler × 5 lần) và 5 lần ProSmart HPA. Cột "Smart tốt hơn" là số lần chạy (theo cặp cùng số thứ tự) Smart HPA tốt hơn K8s HPA; p là kiểm định Mann-Whitney U chính xác, hai phía (với 5 lần mỗi bên, p nhỏ nhất có thể là 0,008).

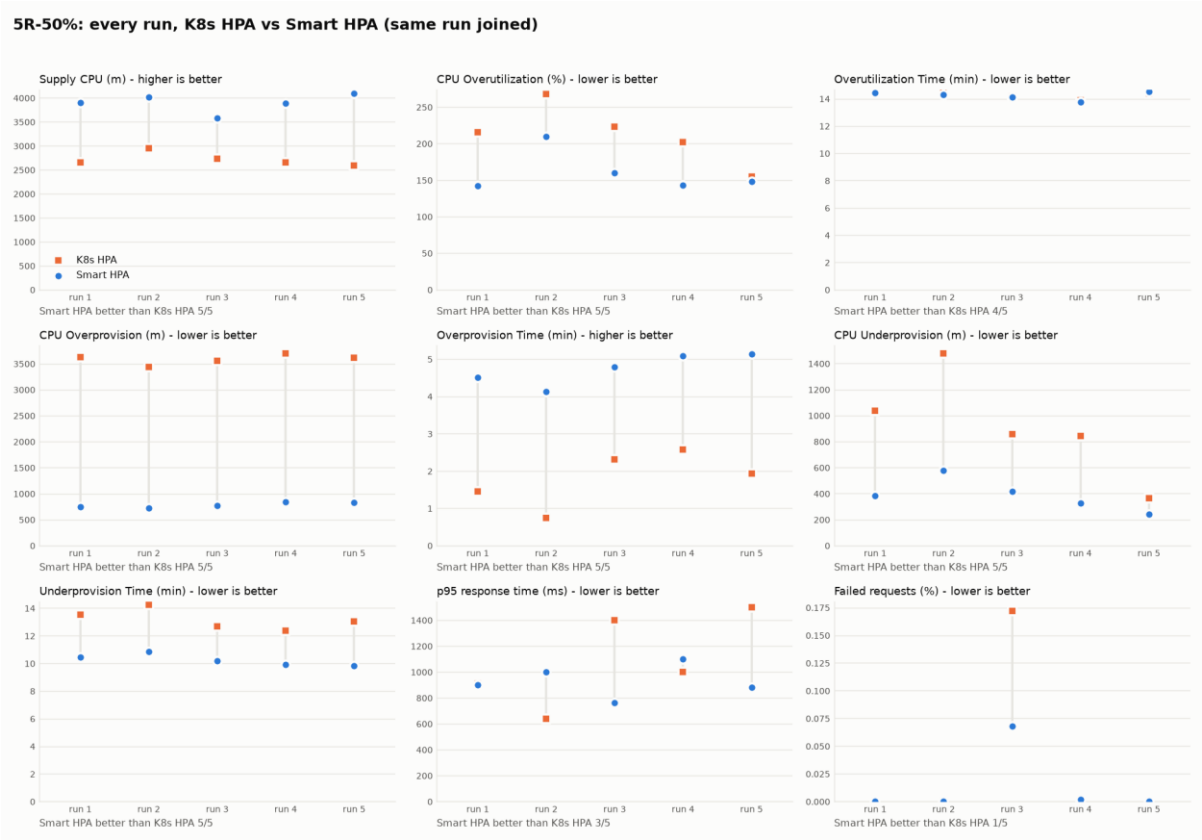
5.1. Kịch bản 5R-50%

Bảng 5.1: Kết quả 5R-50% (TB ± ĐLC, n = 5 mỗi bên)

Chỉ số	K8s HPA	Smart HPA	Smart tốt hơn	p
CPU Underprovision (m)	916 ± 399	389 ± 124	5/5	0,056
Underprovision Time (phút)	13,17 ± 0,74	10,25 ± 0,42	5/5	0,008
CPU Overprovision (m)	3.588 ± 95	781 ± 52	5/5	0,008
Overprovision Time (phút)	1,81 ± 0,73	4,73 ± 0,42	5/5	0,008
Supply CPU (m)	2.718 ± 139	3.898 ± 196	5/5	0,008
CPU Overutilization (%)	213 ± 41	160 ± 28	5/5	0,056
Overutilization Time (phút)	14,32 ± 0,28	14,23 ± 0,30	4/5	0,690
p95 latency (ms)	1.090 ± 356	928 ± 129	3/5	0,460

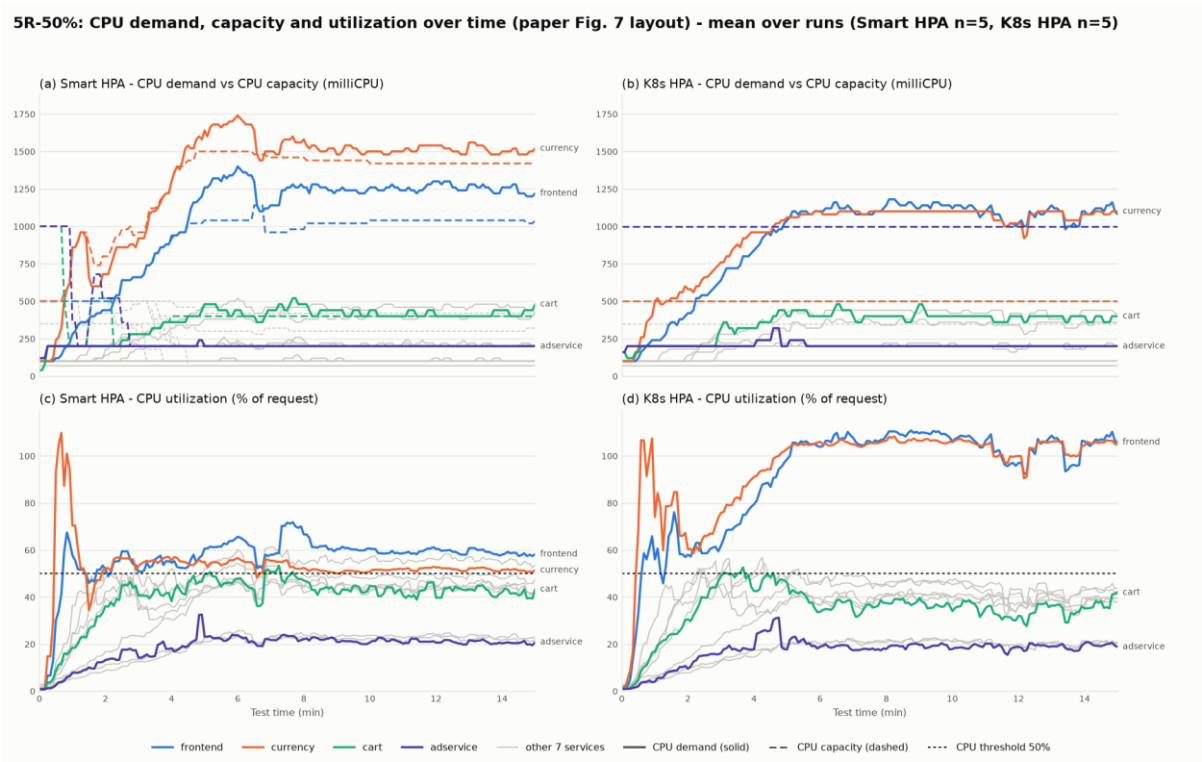
Bảng 5.2: CPU Underprovision và Supply CPU theo từng lần chạy, 5R-50%

Lần	Underprov. K8s HPA	Underprov. Smart HPA	Supply K8s HPA	Supply Smart HPA	Đúng hướng
1	1.035m	381m	2.653m	3.901m	✓ / ✓
2	1.476m	577m	2.950m	4.019m	✓ / ✓
3	858m	418m	2.738m	3.582m	✓ / ✓
4	843m	327m	2.655m	3.891m	✓ / ✓
5	367m	242m	2.596m	4.096m	✓ / ✓



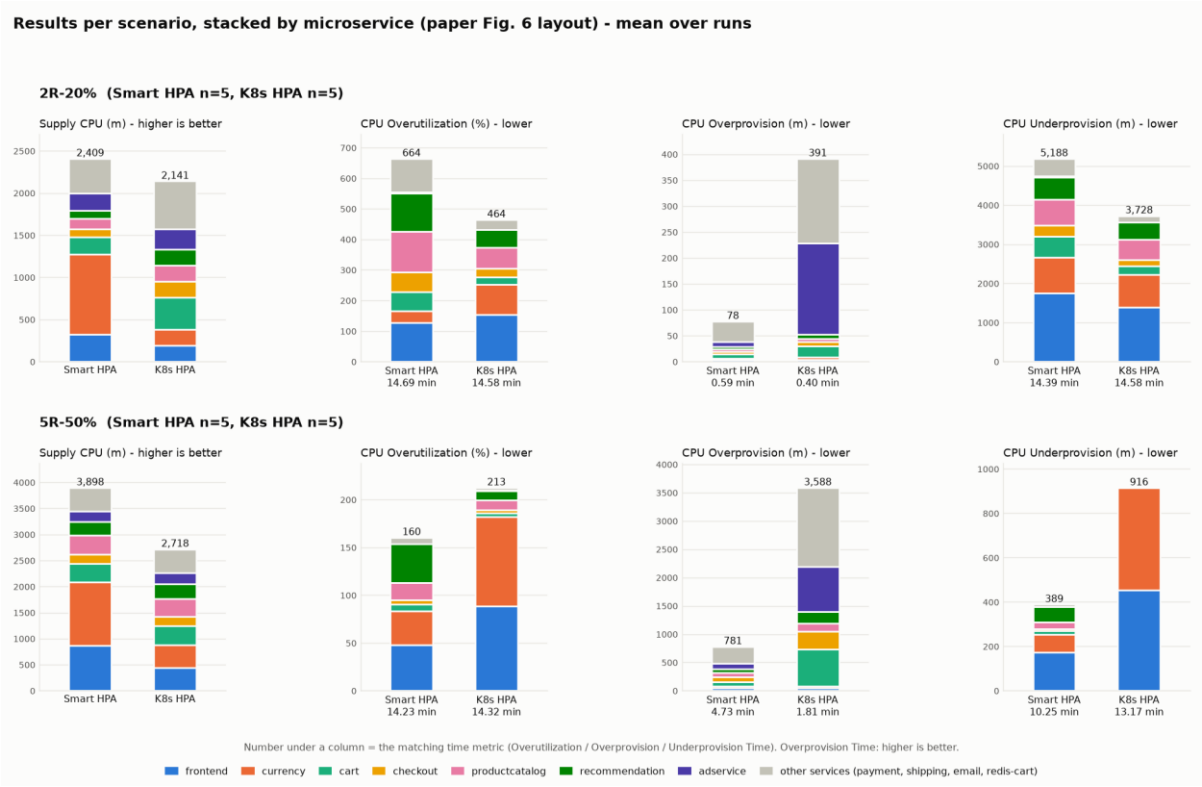
Hình 5.1: Các chỉ số theo từng lần chạy, 5R-50%. Mỗi lần chạy có một điểm K8s HPA và một điểm Smart HPA.

Nhận xét. Smart HPA tốt hơn K8s HPA ở 6/7 chỉ số của bài báo trong cả 5 lần chạy; Overutilization Time đúng hướng 4/5 lần (lần 5: 14,52 so với 14,41 phút, gần như ngang). Xu hướng khớp bài báo và **lặp lại ở mọi lần chạy**. Hành vi chuyển quota cũng giống Fig. 7 của bài báo: khoảng phút thứ 1–2, `frontend` và `currencyservice` vượt maxR 5 và nhận quota từ `adservice`, `cartservice`, `emailservice`, `shippingservice`… (`frontend` lên tới 10 replica, `currencyservice` 14 replica), trong khi K8s HPA giữ hai service này ở 5 replica dù cần khoảng 1.200m CPU.



Hình 5.2: Demand/capacity và CPU theo thời gian (trung bình các lần chạy), 5R-50%, bố cục theo Fig. 7 của bài báo.

Khác bài báo. (i) Smart HPA vẫn còn Underprovision 389m (bài báo: 0m). Mục 6 (W9) và mục 7.2 chỉ ra nguyên nhân: code ARM làm mất dần quota trong lúc chạy; sửa lỗi này thì Underprovision còn 10m. (ii) Overutilization Time gần ngang nhau (bài báo: giảm 1,98 lần). (iii) p95 latency chỉ tốt hơn 3/5 lần, không có ý nghĩa thống kê.

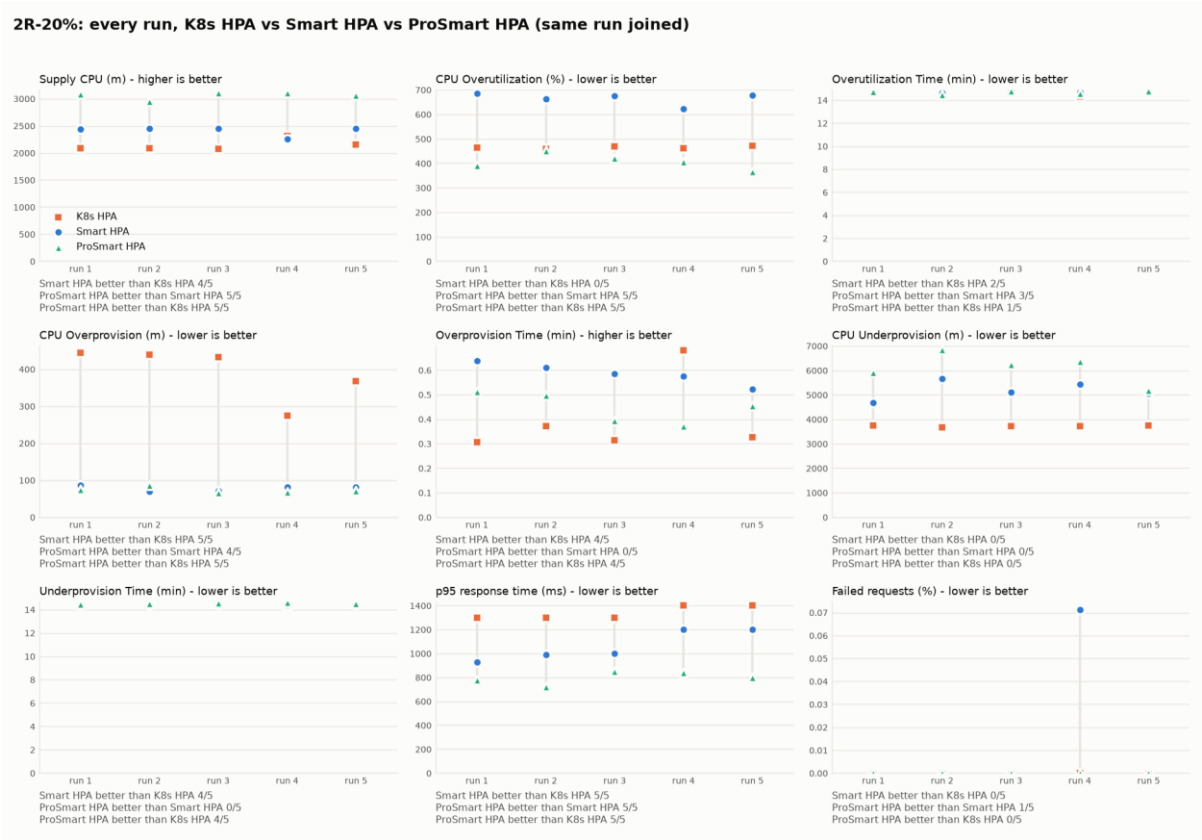


Hình 5.3: Các chỉ số tách theo từng microservice (cột chồng), cả hai kịch bản, bố cục theo Fig. 6 của bài báo.

5.2. Kịch bản 2R-20%

Bảng 5.3: Kết quả 2R-20% (TB ± ĐLC, n = 5 mỗi bên)

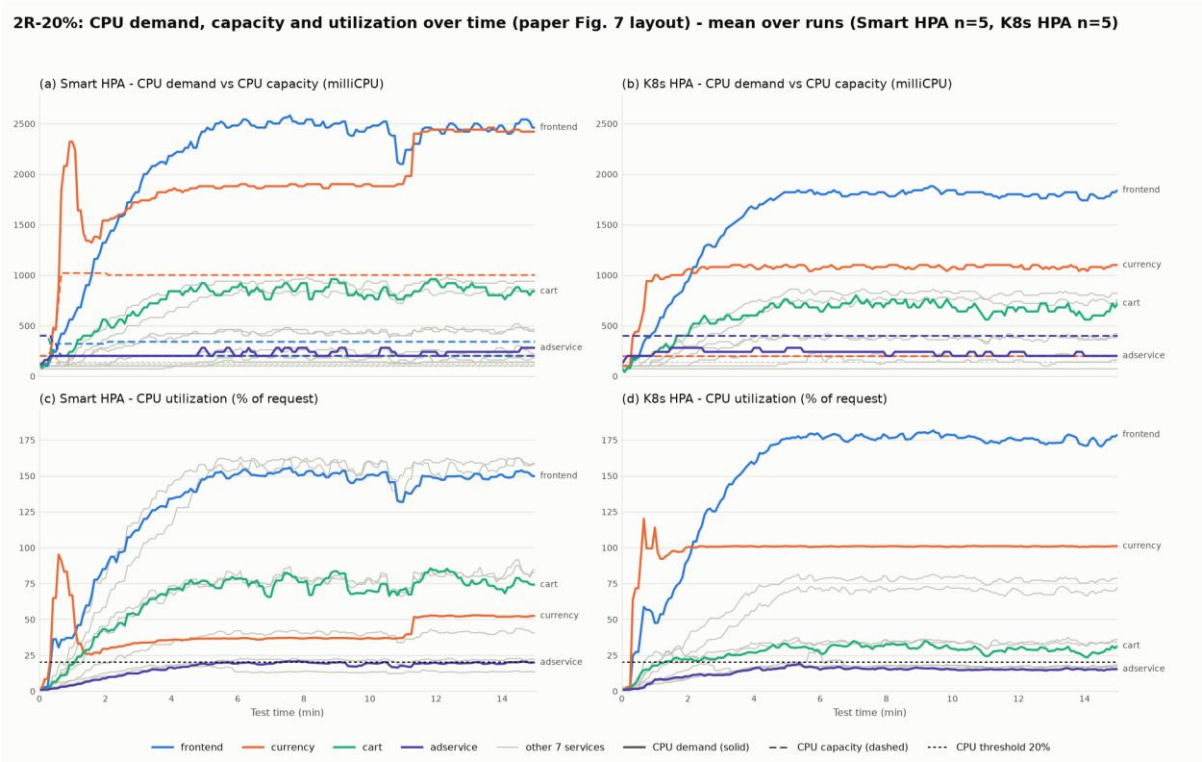
Chỉ số	K8s HPA	Smart HPA	Smart tốt hơn	p
CPU Underprovision (m)	3.728	5.188	0/5	0,008
Underprovision Time (phút)	14,58	14,39	4/5	0,151
CPU Overprovision (m)	391	78	5/5	0,008
Overprovision Time (phút)	0,40	0,58	4/5	0,151
Supply CPU (m)	2.141	2.409	4/5	0,016
CPU Overutilization (%)	464	664	0/5	0,008
Overutilization Time (phút)	14,58	14,69	2/5	0,690
p95 latency (ms)	1.340	1.064	5/5	0,008



Hình 5.4: Các chỉ số theo từng lần chạy, 2R-20%.

Nhận xét. Kết quả **không "gần ngang nhau"** như bài báo. Smart HPA tốt hơn ở Overprovision và p95 latency (5/5 lần), nhưng **kém hơn ở CPU Underprovision và CPU Overutilization trong cả 5/5 lần**. Vì vậy ở kịch bản này, "nhất quán" được hiểu là với từng chỉ số, hướng chênh lệch giống nhau ở mọi lần chạy, kể cả các chỉ số Smart HPA kém hơn.

Nguyên nhân. Không phải do thiếu tài nguyên máy: không có pod `Unschedulable` ở mọi lần chạy, supply chỉ khoảng 2,4/12 vCPU. Lỗi ARM dòng 101 cũng không ảnh hưởng ở maxR = 2. Nguyên nhân là ARM chia quota **tham lam** lúc tải tăng (khoảng phút 1): `currencyservice` được đẩy lên maxR 12, `productcatalogservice` và `recommendationservice` bị lấy quota còn 1 replica (supply từ ~190m xuống 100m) rồi quá tải (CPU ~175%) và giữ nguyên như vậy đến hết lần chạy (Hình 5.5). Đây chính là điểm yếu W4: Algorithm 2 chỉ xét lượng thiếu tại thời điểm chia, không xét vai trò của service.



Hình 5.5: Demand/capacity và CPU theo thời gian, 2R-20%.

Lần chạy ngược hướng. Run 4 (cặp đầu tiên sau khi bật lại VM) ngược hướng ở Supply CPU, Overprovision Time và Underprovision Time: K8s HPA scale `adservice` lên 2 replica (supply 2.309m so với 2.077–2.150m ở các lần khác), còn ARM của Smart HPA chia quota khác các lần khác (`currencyservice` 664m so với 855–1.129m). Lần này không có pod `Unschedulable` và không mất ping nào nên được giữ lại.

5.3. ProSmart HPA (2R-20%, window 25 giây)

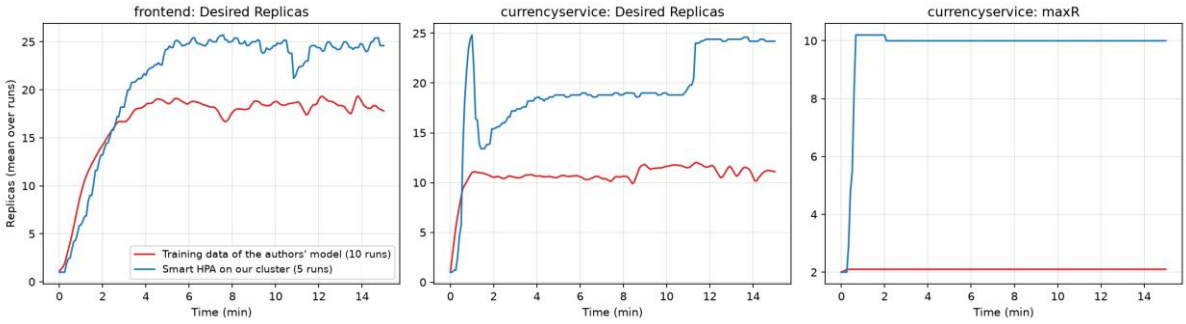
Vì sao nhóm train lại model. Theo mục 5.1 của bài báo, 22 model PROPHET được train từ Knowledge Base của các lần chạy Smart HPA. Dữ liệu train được lưu ngay trong 22 file model của tác giả; nhóm đọc ra và thấy: (i) có 1.810 dòng = 10 lần chạy × 181 điểm (lưới 5 giây, 0–900 giây); (ii) tổng maxR của 11 service bằng 22 ở cả 1.810 dòng, trong khi 1.800 dòng có service có maxR khác 2 (emailservice nhường quota cho frontend hoặc currencyservice). Tổng quota không đổi nhưng quota chuyển giữa các service là cơ chế của Smart HPA (K8s HPA giữ maxR cố định), nên đây là Knowledge Base Smart HPA trên cluster của tác giả. (iii) Model chỉ dùng thời gian làm đầu vào (W7), nên dự đoán chỉ phụ thuộc vào thời điểm trong lần chạy, tức là lặp lại đường cong replica trên cluster của tác giả.

Bảng 5.4 và Hình 5.6 so dữ liệu đó với Knowledge Base 5 lần chạy Smart HPA 2R-20% trên cluster của nhóm. Ở pha tải ổn định (từ phút 5), 9/11 service của nhóm cần nhiều replica hơn 15–89%, adservice cần ít hơn 41%. maxR trong dữ liệu tác giả chỉ từ 1 đến

3, của nhóm từ 1 đến 12 (currencyservice), tức là ARM trên cluster nhóm chuyển quota mạnh hơn nhiều.

Bảng 5.4: Dữ liệu train của model tác giả so với Knowledge Base Smart HPA của nhóm, 2R-20% (DR: số replica mong muốn TB từ phút 5)

Service	DR tác giả	DR nhóm	Chênh lệch	maxR tác giả	maxR nhóm
adservice	2,0	1,2	−41%	2	1–2
cartservice	3,7	4,3	+15%	2	1–2
checkoutservice	2,9	4,4	+54%	2	1–2
currencyservice	11,0	20,9	+89%	2–3	2–12
emailservice	1,4	1,7	+26%	1–2	1–2
frontend	18,3	24,6	+34%	2–3	2–5
paymentservice	3,0	4,4	+47%	2	1–2
productcatalogservice	7,2	9,3	+30%	2	1–2
recommendationservice	7,2	8,2	+15%	2	1–2
redis-cart	2,0	2,0	−1%	2	2
shippingservice	2,0	2,5	+27%	2	1–2



Hình 5.6: Dữ liệu train của model tác giả (đỏ, 10 lần chạy) và Knowledge Base Smart HPA của nhóm (xanh, 5 lần chạy), 2R-20%: Desired Replicas của frontend, currencyservice và maxR của currencyservice.

Model của tác giả trên dữ liệu của nhóm. Nhóm cho cả hai bộ model dự đoán giá trị lúc t + 25 giây trên 2 lần chạy Smart HPA không dùng để train (run 4–5). Model nhu cầu của tác giả sai trung bình 1,93 replica và đoán thiếu 42% số lần; model của nhóm sai 0,96 replica và đoán thiếu 15%. Với model capacity là 1,23 so với 0,42 replica. Cách đoán "25 giây sau giống bây giờ" vẫn có sai số trung bình thấp nhất (0,58 và 0,04 replica) nhưng đoán thiếu nhu cầu 23% số lần. Vì vậy nhóm làm lại đúng quy trình của bài báo trên cluster của nhóm thay vì dùng model của tác giả. So sánh này là offline; nhóm chưa chạy ProSmart trên cluster với model của tác giả để đối chiếu trực tiếp.

Trước khi chạy trên cluster, nhóm cũng kiểm tra chéo model của nhóm (train trên 2 trong 3 lần chạy dùng để train, thử trên lần còn lại). Sai số trung bình của PROPHET (0,84 replica) lớn hơn cách đoán đơn giản "25 giây sau giống bây giờ" (0,47), nhưng

PROPHET ít đoán thiếu hơn hẳn lúc tải tăng (15% so với 33% số lần), phù hợp mục đích scale sớm.

Bảng 5.5: ProSmart HPA so với Smart HPA và K8s HPA (TB 5 lần)

Chỉ số	K8s HPA	Smart HPA	ProSmart HPA	ProSmart so với Smart	Bài báo
Supply CPU (m)	2.141	2.409	3.065	+27,2% (5/5)	+2,17%
CPU Overutilization (%)	464	664	408	−38,6% (5/5)	−25,40%
CPU Overprovision (m)	391	78	73	−6,1% (4/5)	−28,24%
CPU Underprovision (m)	3.728	5.188	6.111	+17,8% (0/5)	−25,34%
p95 latency (ms)	1.340	1.064	798	−25,0% (5/5)	không đo

Khớp bài báo: Overutilization giảm nhất quán 5/5 lần; p95 latency tốt nhất trong ba auto-scaler. **Lệch bài báo:** Underprovision tăng thay vì giảm, Overprovision gần như không giảm.

Lý do chính (W8): supply của ProSmart là 2.953–3.114m, **vượt tổng quota 2.540m** của kịch bản 2R (+16–23%) ở cả 5 lần. Algorithm 4 (dòng 19–20: `ResSD = No Scale, ResDR = CR`) giữ nguyên replica của service vừa bị ARM hạ capacity xuống dưới số replica hiện tại, trong khi phần quota đó đã cấp cho service khác. Phần CPU thêm này giúp Overutilization và latency; đồng thời capacity bị hạ nhưng pod không giảm nên chỉ số Underprovision (= demand − capacity) tăng. Do đó **ProSmart tốt hơn ở một số chỉ số nhưng không cùng mức tài nguyên**; tuyên bố "resource-efficient" không được xác nhận trên cluster của nhóm. Run 4–5 chạy xen kẽ cùng ngày với hai auto-scaler kia cho cùng kết luận, nên chênh lệch không do khác ngày chạy.

5.4. Tổng hợp tính nhất quán

Bảng 5.6: Số lần đúng hướng của Smart HPA so với K8s HPA

Chỉ số	5R-50%	2R-20%
CPU Underprovision	5/5 tốt hơn	5/5 kém hơn
Underprovision Time	5/5 tốt hơn	4/5 tốt hơn
CPU Overprovision	5/5 tốt hơn	5/5 tốt hơn
Overprovision Time	5/5 tốt hơn	4/5 tốt hơn
Supply CPU	5/5 tốt hơn	4/5 tốt hơn
CPU Overutilization	5/5 tốt hơn	5/5 kém hơn
Overutilization Time	4/5 (gần ngang)	2/5 (gần ngang)

Ở 5R-50%, 6/7 chỉ số nhất quán tuyệt đối; ở 2R-20%, 4/7 chỉ số nhất quán tuyệt đối (2 trong số đó theo hướng Smart HPA kém hơn), 3 chỉ số còn lại 4/5 do một lần chạy (run

4) có phân bổ quota khác thường. Hai chỉ số “thời gian” dao động quanh mức gần 15 phút ở cả hai bên, nên chênh lệch nhỏ và không ổn định giữa các lần.

6. Phân tích điểm yếu của bài báo

Bảng 6.1: Các điểm yếu phát hiện được

#	Điểm yếu	Bằng chứng
W1	Không đo chỉ số phía người dùng (latency, lỗi, vi phạm SLO)	Chỉ có chỉ số CPU ở Table 1; mục 6 của bài tự nhận là hướng tương lai
W2	Chỉ thử một kiểu tải (tăng dần rồi giữ cao)	Mục 7 (Threats to validity) thừa nhận chưa thử tải bursty
W3	Baseline có thể chưa công bằng: K8s HPA bị chặn maxR riêng, Smart HPA dùng chung "quỹ" quota	Algorithm 2
W4	Chia quota tham lam theo lượng thiếu, không xét tầm quan trọng của service	Algorithm 2 dòng 17; kết quả 2R-20% (mục 5.2)
W5	Tuyên bố giảm communication overhead nhưng không đo	Mục 1, mục 3.2 của bài
W6	Chỉ báo cáo trung bình, không có độ lệch chuẩn hay kiểm định thống kê	Mục 5.2 của bài
W7	Replication package ProSmart không tái hiện được trực tiếp	22 model PROPHET đi kèm không có biến hồi quy nào, trái mô tả mục 5.1; code có ít nhất 7 lỗi
W8	ProSmart có thể dùng vượt tổng quota	Algorithm 4 dòng 19–20; supply vượt quota 16–23% ở cả 5 lần (mục 5.3)
W9	Code ARM của Smart HPA làm mất quota, trái ý "tổng quota không đổi"	5R-50%: tổng capacity từ 6.350m giảm còn 3.970–4.770m (−25–37%) ở cả 5 lần

Ngoài ra, mô tả tải trong bài tự mâu thuẫn: "600 user, mỗi 2 giây thêm 1 user, đủ 600 sau 5 phút" (300 giây / 2 giây chỉ được 150 user), trong khi Locust `spawn rate 2` nghĩa là 2 user/giây. Smart HPA cũng không có cửa sổ ổn định khi scale down (K8s HPA có 300 giây), và pha Monitor chờ đến khi `kubectl top` thấy đủ số pod nên cả vòng điều khiển bị chặn lúc pod đang khởi động/tắt.

Nhóm chọn ba cải tiến: **C1** giải quyết W1 (và W6), **C2** giải quyết W2, **C3** giải quyết W4. Trong quá trình làm C3, nhóm phát hiện thêm W9.

7. Đề xuất cải tiến và kết quả

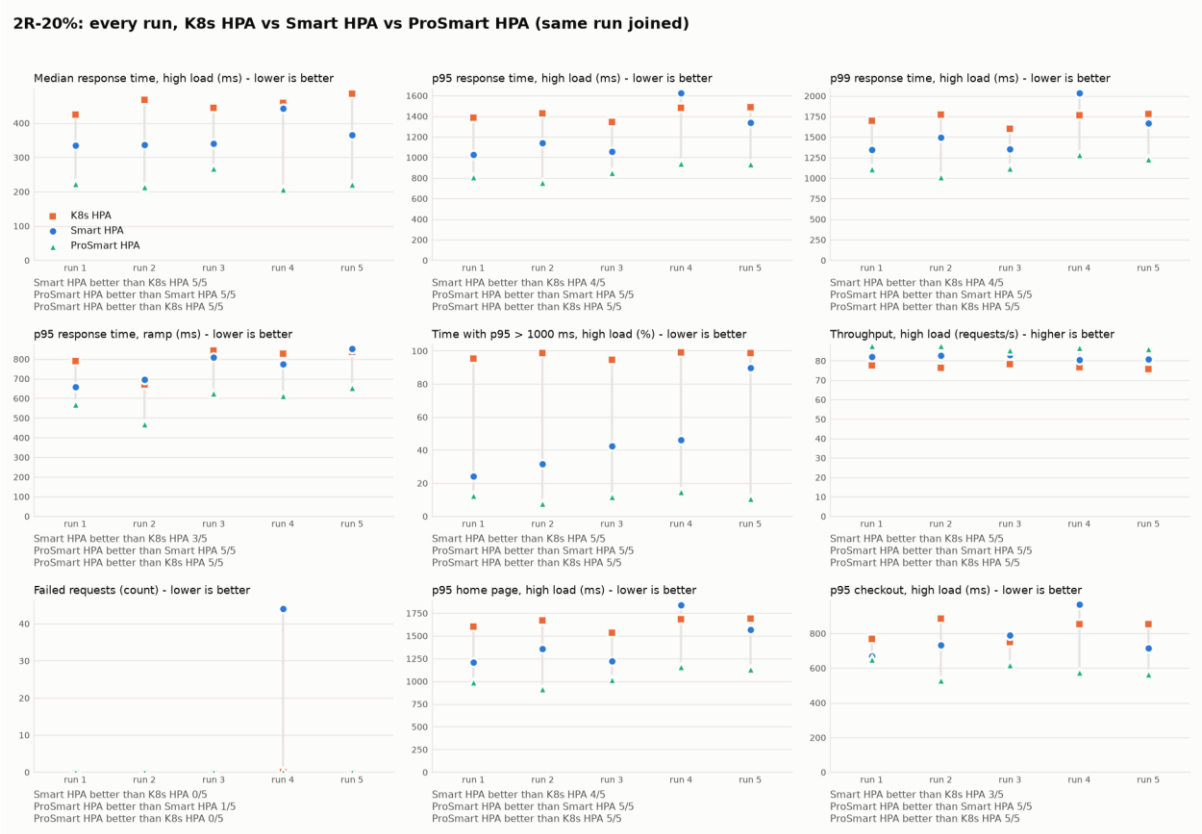
7.1. C1: Đo chỉ số phía người dùng

Cách làm. Nhóm tính thêm chỉ số phía người dùng từ lịch sử thống kê của Locust (mỗi giây một dòng, percentile tính trên 10 giây gần nhất), chia pha tăng tải (phút 0–5) và pha tải cao (phút 5–15), tính 9 chỉ số: median/p95/p99 pha tải cao, p95 pha tăng tải, % thời gian vi phạm SLO, throughput, số request lỗi, p95 trang chủ và p95 checkout. Ngưỡng SLO chốt là **p95 ≤ 1.000 ms**; ngưỡng 500 ms dự kiến ban đầu bị vi phạm 98–100% thời gian ở mọi lần chạy nên không phân biệt được auto-scaler. Cải tiến này không cần chạy thêm thí nghiệm. Đồng thời nhóm thêm cột p (Mann-Whitney U) cho cả 7 chỉ số của bài báo (W6).

Giả thuyết: Smart HPA giảm Underprovision nên latency cũng thấp hơn.

Bảng 7.1: Chỉ số phía người dùng, 2R-20% (TB 5 lần; x/5 là số lần tốt hơn K8s HPA)

Chỉ số (pha tải cao)	K8s HPA	Smart HPA	ProSmart HPA
Median (ms)	456	364 (5/5, p = 0,016)	227 (5/5, p = 0,008)
p95 (ms)	1.426	1.238 (4/5, p = 0,151)	859 (5/5, p = 0,008)
% thời gian p95 > 1.000 ms	97,3	46,7 (5/5, p = 0,008)	11,6 (5/5, p = 0,008)
Throughput (request/s)	76,9	81,8 (5/5, p = 0,008)	86,7 (5/5, p = 0,008)
Request lỗi (tổng 5 lần)	0	44 (chỉ ở run 4)	0



Hình 7.1: Chỉ số phía người dùng theo từng lần chạy, 2R-20%.

Bảng 7.2: Chỉ số phía người dùng, 5R-50% (TB ± ĐLC)

Chỉ số (pha tải cao)	K8s HPA	Smart HPA	Smart tốt hơn	p
p95 (ms)	1.397 ± 767	1.085 ± 151	3/5	0,841
% thời gian p95 > 1.000 ms	51,1 ± 40,3	30,7 ± 16,8	2/5 (1 lần bằng)	0,595
Throughput (request/s)	81,4 ± 6,0	84,7 ± 1,0	3/5	0,421

Kết luận C1. Ở 2R-20%, thứ tự ProSmart → Smart HPA → K8s HPA lặp lại ở 8/9 chỉ số; Smart HPA tốt hơn K8s HPA về phía người dùng **dù kém hơn ở CPU Underprovision và Overutilization**. Nghĩa là chỉ số CPU của bài báo và trải nghiệm người dùng không đi cùng hướng, đúng điểm yếu W1. Ở 5R-50%, giả thuyết **không được xác nhận**: không chỉ số phía người dùng nào nhất quán (cao nhất 3/5, mọi $p \geq 0,42$) dù 6/7 chỉ số CPU nhất quán 5/5; Smart HPA chỉ ổn định hơn giữa các lần (ĐLC p95 151 so với 767 ms).

7.2. C3: Chia quota theo độ ưu tiên

Ý tưởng. Sửa Algorithm 2 để ARM xét vai trò của service trong luồng người dùng thay vì chỉ lượng thiếu. Code mới viết thành một module riêng, ARM gốc giữ nguyên. Bốn điểm khác ARM gốc:

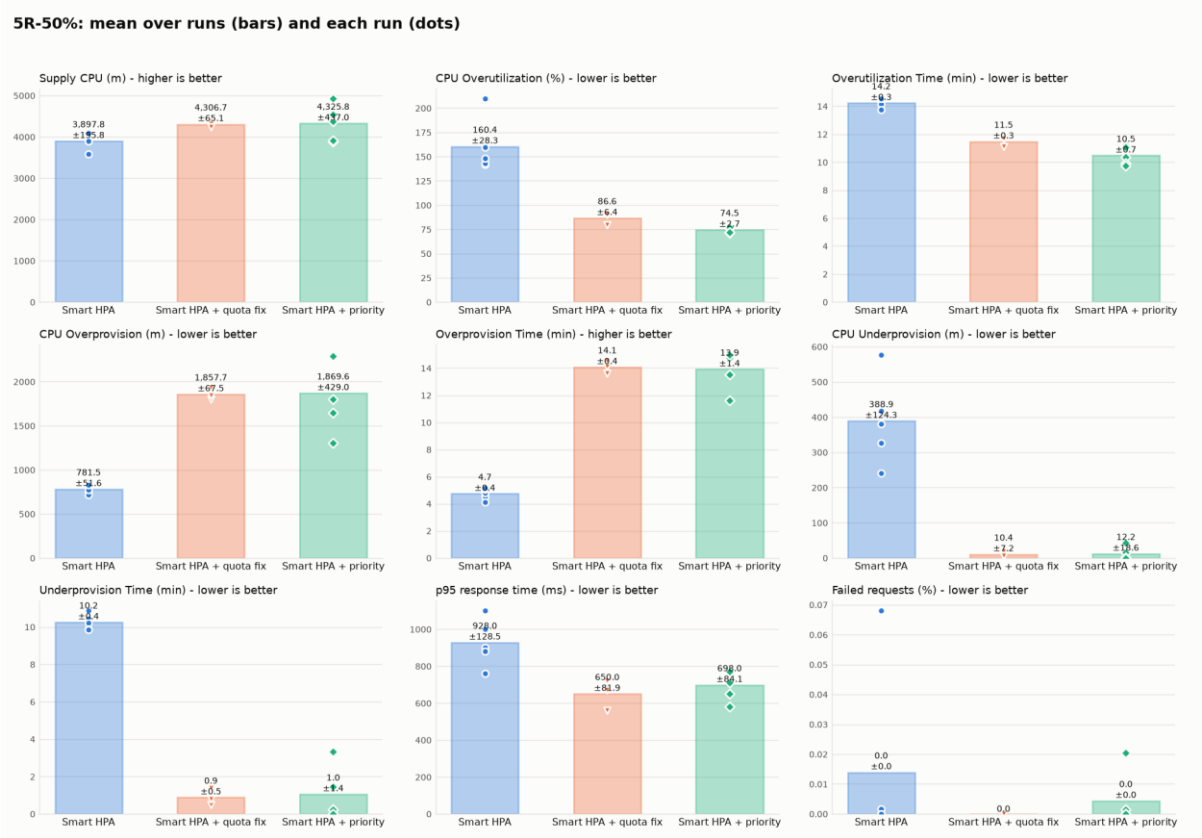
1. Service thiếu được xếp theo **trọng số × CPU còn thiếu**, với trọng số `frontend` = 3; `checkout`, `cart`, `productcatalog`, `currency` = 2; còn lại = 1.
2. Service thừa có trọng số ≥ 2 giữ lại 1 replica dự phòng, không đưa vào quỹ chung.
3. Quota còn lại trả về service trọng số cao trước.
4. Mỗi vòng chia lại từ **tổng quota của ứng dụng**, nên phần quota lẻ chưa trả được ở vòng trước được chia tiếp ở vòng sau. Điểm này sửa W9, không thuộc ý tưởng ưu tiên.

Kiểm tra offline: 2.000 kịch bản ngẫu nhiên × 300 vòng, tổng capacity không lần nào vượt quota. Để tách tác dụng của điểm 4 khỏi ý tưởng ưu tiên, nhóm chạy thêm **biến thể đối chứng**: cùng code nhưng mọi trọng số = 1 và không dự phòng (chỉ còn điểm 4), chạy xen kẽ với C3 cùng ngày.

Bảng 7.3: Kết quả C3 ở 5R-50% (TB ± ĐLC)

Chỉ số	Smart HPA (n=5)	Đối chứng: chỉ sửa W9 (n=3)	C3: sửa W9 + ưu tiên (n=5)	p (C3 vs đối chứng)
CPU Underprovision (m)	389 ± 124	**10 ± 7**	12 ± 19	0,571
Underprovision Time (phút)	10,25 ± 0,42	**0,88 ± 0,45**	1,05 ± 1,39	0,786
CPU Overutilization (%)	160 ± 28	87 ± 6	**75 ± 3**	0,036
Overutilization Time (phút)	14,23 ± 0,30	11,47 ± 0,33	**10,47 ± 0,66**	0,071

Chỉ số	Smart HPA (n=5)	Đối chứng: chỉ sửa W9 (n=3)	C3: sửa W9 + ưu tiên (n=5)	p (C3 vs đối chứng)
Supply CPU (m)	3.898 ± 196	4.307 ± 65	4.326 ± 437	0,786
CPU Overprovision (m)	**781 ± 52**	1.858 ± 67	1.870 ± 429	0,786
p95 pha tải cao (ms)	1.085 ± 151	**653 ± 74**	812 ± 310	0,571
Throughput (request/s)	84,7 ± 1,0	**89,0 ± 0,7**	87,5 ± 2,6	0,786



Hình 7.2: Smart HPA gốc, đối chứng (chỉ sửa W9) và C3 ở 5R-50%.

- **So với Smart HPA gốc**, C3 tốt hơn 5/5 lần ở Underprovision, Underprovision Time, Overutilization, Overutilization Time, Overprovision Time và p95 (p = 0,008); kém hơn 5/5 ở CPU Overprovision.
- **Phần lớn cải thiện là do sửa W9, không phải do ưu tiên.** Đối chứng đã đưa Underprovision từ 389m xuống 10m, p95 từ 1.085 xuống 653 ms, nhất quán 3/3 lần. Tổng capacity cuối lần chạy của Smart HPA gốc chỉ còn 3.970–4.770m, của đối chứng và C3 là 6.310–6.350m (quota 6.350m); có thêm 1.600–2.400m để chia nên `frontend` lên 14–16 replica, `currency` 17–23. Overprovision cao hơn cũng vì capacity lớn hơn.
- **Ưu tiên chỉ giảm thêm Overutilization:** 87% → 75% (cả 5 lần C3 thấp hơn cả 3 lần đối chứng, p = 0,036). Phía người dùng không tốt hơn đối chứng.
- Smart HPA sau khi sửa W9 đạt Underprovision gần 0, **đúng con số bài báo công bố** (0m) mà code gốc trong replication package không đạt được.

Bảng 7.4: Kết quả C3 ở 2R-20% (C3 n = 3)

Chỉ số	Smart HPA	Smart HPA + ưu tiên	C3 tốt hơn	p
CPU Overutilization (%)	664 ± 25	608 ± 17	3/3	0,071
CPU Underprovision (m)	5.188 ± 371	5.502 ± 1.061	2/3	1,000
Median pha tải cao (ms)	364 ± 46	412 ± 17	0/3	0,250
p95 pha tải cao (ms)	1.238 ± 247	1.172 ± 85	0/3	1,000
Throughput (request/s)	81,8 ± 1,1	80,3 ± 0,7	0/3	0,143

Ở 2R-20%, Smart HPA gốc không mất quota nên đây là so sánh sạch của quy tắc ưu tiên, và giả thuyết **không được xác nhận** (mọi p > 0,05). Nguyên nhân: ưu tiên chỉ có tác dụng khi nhiều service thiếu **cùng lúc**. Ở run 2 và 3, `currency` (trọng số 2) thiếu trước trong khoảng 20 giây đầu và lấy gần hết quỹ (lên 8 replica); `frontend` (trọng số 3) thiếu muộn hơn và kẹt ở 2 replica với CPU 180–192% trong khi cần 18–20. Kết quả phụ thuộc thứ tự service nào thiếu trước, tức W4 vẫn chưa được giải quyết. Hướng sửa tiếp theo: cho service trọng số cao lấy lại phần quota mà service trọng số thấp đã mượn vượt maxR ban đầu, có giới hạn để không làm service kia thiếu ngược lại.

Lưu ý: C3 chạy khác ngày với Smart HPA và K8s HPA. Chênh lệch nhỏ ở 2R-20% vì vậy không đủ tin cậy; chênh lệch lớn ở 5R-50% ít bị ảnh hưởng, và lần đối chứng đã chạy xen kẽ cùng ngày với C3. C3 run 4 có hai lần chập mạng khoảng 5 giây (13/63.765 request lỗi) nên số phía người dùng của lần này không dùng để kết luận.

7.3. C2: Tải đột biến

Ý tưởng. Bài báo chỉ thử một kiểu tải tăng dần rồi giữ cao (W2). C2 định nghĩa một kịch bản tải riêng trong Locust để số user nhảy giữa mức nền và đỉnh, kiểm tra xem Smart HPA có chuyển quota qua lại liên tục (flapping) hay không.

Bảng 7.5: Kịch bản tải đột biến (tổng 15 phút)

Thời gian	Số user	Tốc độ sinh user	Pha
0–3 phút	50	5/s	Tải nền
3–7 phút	250	50/s	Đột biến 1
7–10 phút	50	50/s	Giảm đột ngột
10–13 phút	250	50/s	Đột biến 2
13–15 phút	50	50/s	Giảm

Chỉ số ổn định. Nhóm đếm từ Knowledge Base: số lần đổi replica, số lần đảo chiều (scale up rồi scale down cùng service hoặc ngược lại), số lần đổi maxR (chuyển quota) và biên độ dao động replica lớn nhất của một service. Áp dụng ngay cho dữ liệu 5R-50% với tải tăng dần đã có:

Bảng 7.6: Độ ổn định khi scale, 5R-50% tải tăng dần (TB ± ĐLC, n = 5)

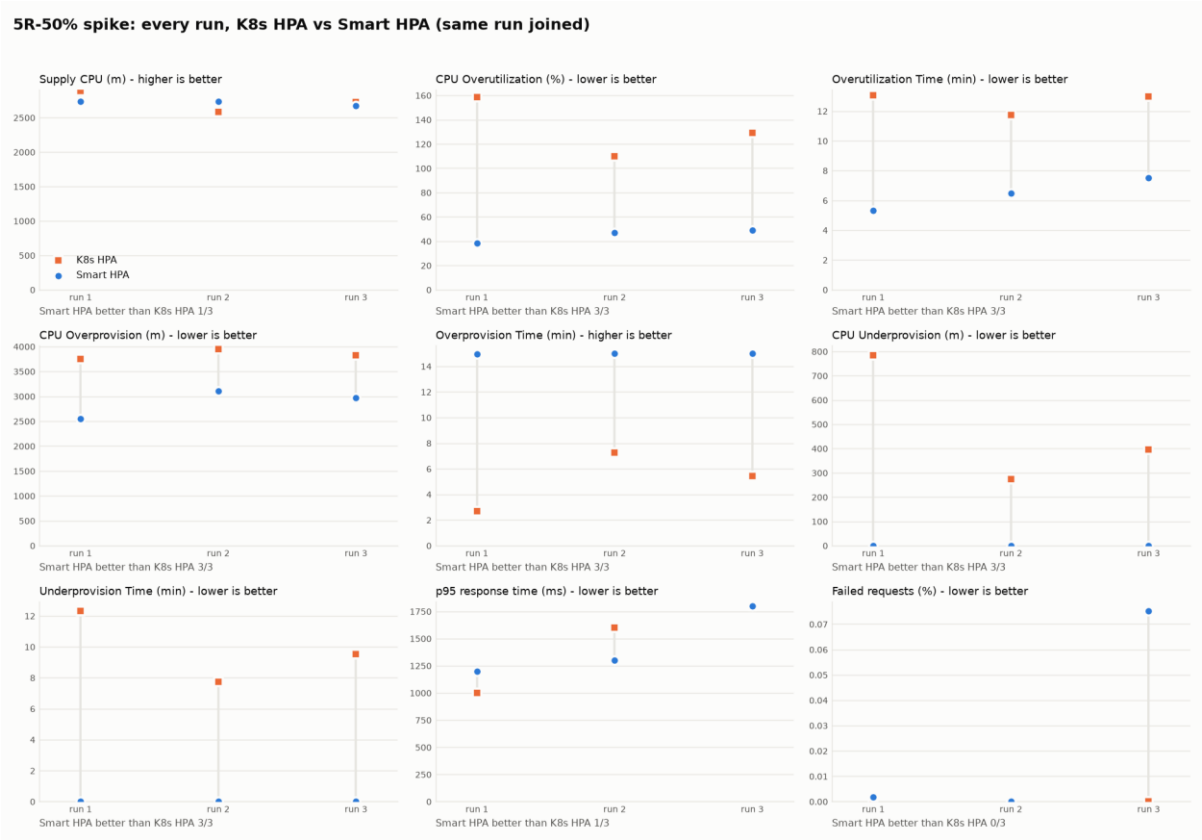
Chỉ số	K8s HPA	Smart HPA	Smart ổn định hơn	p
Số lần đổi replica	16,2 ± 2,4	37,0 ± 11,5	0/5	0,008
Số lần đảo chiều	0,4 ± 0,6	9,0 ± 6,2	0/5	0,008
Số lần chuyển quota	0	31,2 ± 8,1	0/5	0,008
Dao động replica lớn nhất	3,0 ± 0,0	12,4 ± 1,3	0/5	0,008

Ngay cả với tải tăng dần, Smart HPA đổi replica nhiều gấp khoảng 2,3 lần và đảo chiều scale 9 lần mỗi lần chạy (K8s HPA gần như 0), nhất quán 5/5 lần. Nguyên nhân là Smart HPA không có cửa sổ ổn định khi scale down, cộng với việc ARM thay đổi maxR trung bình 31 lần mỗi lần chạy. Với tải đột biến, hiện tượng này dự kiến còn rõ hơn.

Kết quả. Thí nghiệm chạy ở 5R-50%, 3 cặp xen kẽ K8s HPA và Smart HPA. Cả 6 lần hợp lệ ngay lần thử đầu: 0 pod Unschedulable, 0 ping mất trong cả đợt chạy (ping mỗi giây tới 3 node). Với 3 lần mỗi bên, p nhỏ nhất có thể là 0,100, nên không chỉ số nào đạt $p < 0,05$; tính nhất quán được đánh giá chủ yếu bằng số lần đúng hướng.

Bảng 7.7: Kết quả 5R-50% tải đột biến (TB ± ĐLC, n = 3 mỗi bên)

Chỉ số	K8s HPA	Smart HPA	Smart tốt hơn	p
CPU Underprovision (m)	484 ± 267	0 ± 0	3/3	0,100
Underprovision Time (phút)	9,86 ± 2,31	0,00 ± 0,00	3/3	0,100
CPU Overprovision (m)	3.840 ± 100	2.876 ± 289	3/3	0,100
Overprovision Time (phút)	5,12 ± 2,29	14,98 ± 0,01	3/3	0,100
Supply CPU (m)	2.734 ± 153	2.715 ± 39	1/3	1,000
CPU Overutilization (%)	133 ± 24	45 ± 6	3/3	0,100
Overutilization Time (phút)	12,59 ± 0,73	6,43 ± 1,10	3/3	0,100
p95 latency (ms)	1.467 ± 416	1.433 ± 321	1/3 (1 lần bằng)	1,000

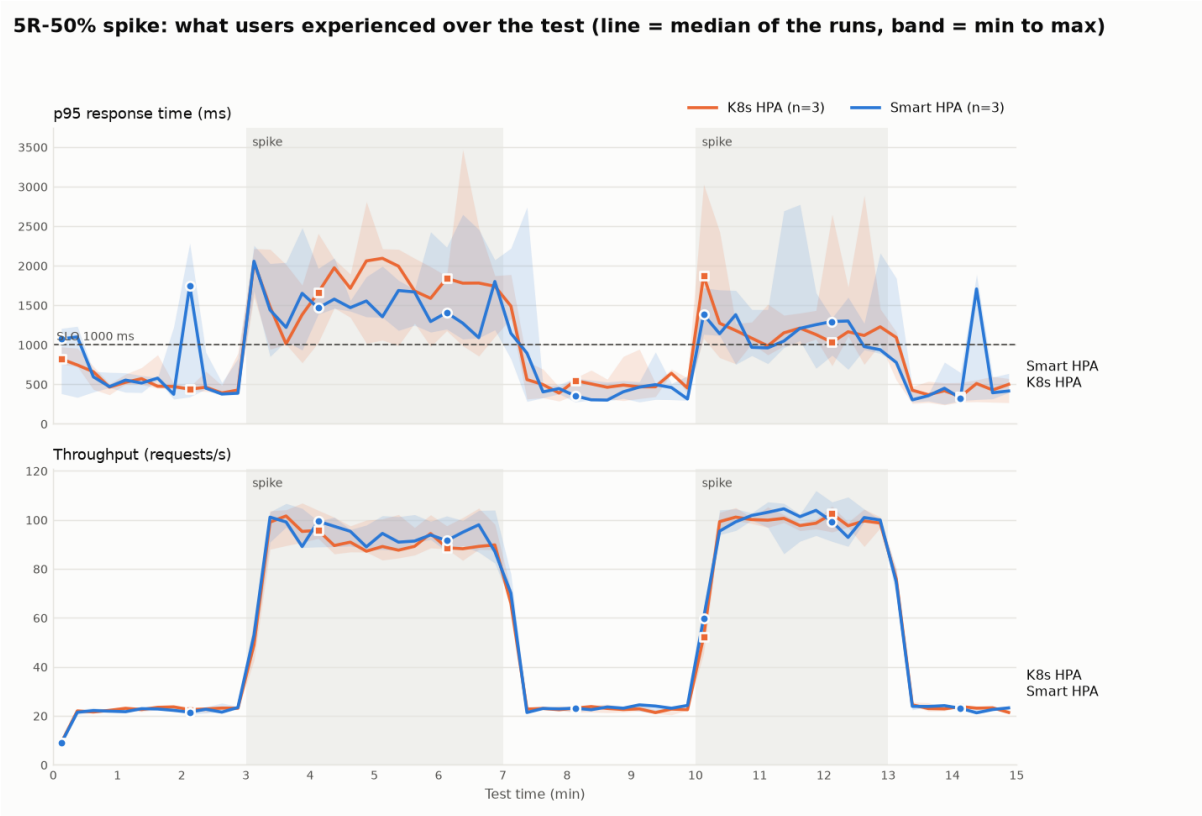


Hình 7.3: Các chỉ số theo từng lần chạy, 5R-50% tải đột biến.

Về tài nguyên, xu hướng của 5R-50% vẫn giữ khi đổi kiểu tải: Smart HPA tốt hơn K8s HPA ở 6/7 chỉ số trong cả 3 lần. Underprovision của Smart HPA bằng 0 ở cả 3 lần (K8s HPA 273–785m), Overutilization giảm khoảng 3 lần (45% so với 133%), Overutilization Time giảm một nửa (6,43 so với 12,59 phút). Supply CPU gần ngang (2.715 so với 2.734m, 1/3 lần). Khác với tải tăng dần (Underprovision 389m, mục 5.1), ở đây Smart HPA không còn Underprovision.

Bảng 7.8: Chỉ số phía người dùng, 5R-50% tải đột biến (TB ± ĐLC, n = 3)

Chỉ số	K8s HPA	Smart HPA	Smart tốt hơn	p
Median pha đột biến (ms)	485 ± 70	481 ± 131	2/3	1,000
p95 pha đột biến (ms)	1.535 ± 418	1.468 ± 291	2/3	1,000
p95 phút đầu của đột biến (ms)	1.480 ± 489	1.457 ± 365	2/3	1,000
p95 tải nền sau đột biến (ms)	548 ± 144	597 ± 130	1/3	1,000
% thời gian p95 > 1.000 ms	74,1 ± 25,6	72,8 ± 18,2	1/3	1,000
Throughput pha đột biến (request/s)	95,5 ± 4,6	96,8 ± 5,0	2/3	1,000
p95 checkout pha đột biến (ms)	951 ± 72	941 ± 189	2/3	1,000
Request lỗi (số request)	0 ± 0	13 ± 22	0/3	0,400



Hình 7.4: p95 và throughput theo thời gian, 5R-50% tải đột biến (đường: trung vị 3 lần, dải: thấp nhất – cao nhất).

Về phía người dùng, hai auto-scaler gần như hòa: không chỉ số nào nhất quán 3/3, mọi p = 1,000 (pha đột biến là phút 3–7 và 10–13, phút đầu của đột biến là phút 3–4 và 10–11, tải nền sau đột biến là phút 7–10 và 13–15). Cả hai vượt SLO 1.000 ms trong khoảng 3/4 thời gian đột biến: p95 nhảy lên 1.400–2.000 ms ngay khi tải tăng và chỉ về hẳn dưới 1.000 ms khi tải giảm (Hình 7.4). Như vậy lợi thế về CPU của Smart HPA một lần nữa không chuyển thành latency tốt hơn, giống kết luận của C1 ở 5R-50%.

Bảng 7.9: Độ ổn định khi scale, 5R-50% tải đột biến (TB ± ĐLC, n = 3)

Chỉ số	K8s HPA	Smart HPA	Smart ổn định hơn	p
Số lần đổi replica	12,3 ± 1,5	89,0 ± 11,4	0/3	0,100
Số lần đảo chiều	0,0 ± 0,0	42,7 ± 10,1	0/3	0,100
Số lần chuyển quota	0	27,3 ± 8,1	0/3	0,100
Dao động replica lớn nhất	2,3 ± 0,6	10,7 ± 0,6	0/3	0,100

Flapping xảy ra rõ rệt. So với tải tăng dần (Bảng 7.6), số lần đổi replica của Smart HPA tăng từ 37 lên 89 và số lần đảo chiều từ 9 lên 43 mỗi lần chạy; một service dao động tới 11 replica. K8s HPA lại ổn định hơn cả khi tải tăng dần (12 lần đổi replica, 0 lần đảo chiều, dao động 2–3 replica): cửa sổ ổn định 300 giây khi scale down dài hơn các pha tải nền (3 phút và 2 phút), nên K8s HPA chỉ scale up và giữ replica qua các pha

giảm tải. Giả thuyết của C2 được xác nhận: khi tải lên xuống liên tục, Smart HPA scale lên xuống và chuyển quota liên tục.

Request lỗi. Smart HPA run 3 có 38/50.569 request lỗi (0,08%), K8s HPA 0 lỗi ở cả 3 lần. Không phải do mạng (0 ping mất). Lỗi rơi vào hai đợt, đều trùng lúc Smart HPA scale down: (1) 32 lỗi mất kết nối (server đóng kết nối giữa chừng) trong 1 giây, đúng lúc `frontend` giảm 5 → 4 trong chuỗi đảo chiều 4 → 5 → 4 → 5 → 4 kéo dài 45 giây; (2) 6 lỗi HTTP 500 ở các trang giỏ hàng, đổi tiền tệ và xem sản phẩm khi Smart HPA thu hẹp mạnh sau đột biến thứ hai (`frontend` 12 → 9 → 5, `productcatalogservice` 4 → 1, `currencyservice` 8 → 6, `cartservice` 2 → 1 trong 30 giây). Nhiều khả năng pod bị xóa khi đang xử lý request; nhóm chưa kiểm tra log pod để xác nhận. Lần chạy vẫn qua bước kiểm tra hợp lệ (lỗi < 1%) nên được giữ lại.

Kết luận C2. Với tải đột biến, Smart HPA vẫn tốt hơn K8s HPA về tài nguyên một cách nhất quán (6/7 chỉ số, 3/3 lần), nhưng không tốt hơn về phía người dùng và bị flapping: đảo chiều scale khoảng 43 lần mỗi lần chạy (K8s HPA 0), và các lần scale down dồn dập trùng với 38/39 request lỗi của Smart HPA. Đây là hệ quả trực tiếp của việc Smart HPA không có cửa sổ ổn định khi scale down (mục 6); thêm cửa sổ này là hướng sửa tự nhiên. Hạn chế: chỉ 3 cặp và một kịch bản (5R-50%).

8. Thảo luận và hạn chế

8.1. Mức độ tái hiện

Xu hướng chính của bài báo ở kịch bản 5R-50% được tái hiện và lặp lại ở mọi lần chạy. Hai chỗ lệch quan trọng đều có nguyên nhân cụ thể: Underprovision khác 0 do W9 (đã chứng minh bằng lần chạy đối chứng), và kết quả 2R-20% không ngang nhau do ARM chia quota tham lam (W4). ProSmart HPA cho kết quả tốt hơn ở Overutilization và latency nhưng vì dùng vượt quota (W8), nên không thể kết luận nó hiệu quả hơn về tài nguyên.

8.2. Hạn chế của bài tập

- **Quy mô nhỏ:** 3 VM trên một laptop, 200 users; các node chia chung CPU vật lý của host nên có thể ảnh hưởng lẫn nhau. Con số tuyệt đối không so trực tiếp được với bài báo.
- **Số lần chạy ít:** 5 lần mỗi cấu hình (bài báo 10), C3 ở 2R-20% chỉ 3 lần, đối chứng 3 lần, C2 chỉ 3 cặp (p nhỏ nhất 0,100). Với n = 5, kiểm định Mann-Whitney chỉ đạt $p < 0,05$ khi hai nhóm gần như không chồng nhau.
- **Ít kịch bản:** 2/9 kịch bản; ProSmart chỉ 1/6 prediction window.
- **Mạng không ổn định:** một số lần chạy bị chập mạng ngắn; đã kiểm soát bằng bước kiểm tra tự động và ghi ping, nhưng p95 vẫn lệch giữa các ngày chạy.
- **ProSmart được cài lại:** do replication package không chạy được, kết quả ProSmart phản ánh cách nhóm hiện thực Algorithm 3–4 và model nhóm tự train trên 3 lần chạy, có thể khác bản của tác giả. Model của tác giả mới được so sánh offline (mục 5.3), chưa chạy trên cluster để đối chiếu trực tiếp.
- **C3 chạy khác ngày** với Smart HPA gốc (trừ lần đối chứng); C2 chỉ thử ở 5R-50% và chưa kiểm tra log pod để xác nhận nguyên nhân request lỗi.

9. Kết luận và hướng phát triển

Nhóm đã dựng cluster k3s 3 node, triển khai Online Boutique và tái hiện thí nghiệm của bài báo với tổng cộng 42 lần chạy hợp lệ (20 lần tái hiện, 5 lần ProSmart, 11 lần C3 và đối chứng, 6 lần C2). Kết quả chính:

1. Ở 5R-50%, Smart HPA tốt hơn K8s HPA ở 6/7 chỉ số trong 5/5 lần chạy, khớp xu hướng bài báo.
2. Ở 2R-20%, kết quả không ngang nhau như bài báo mà Smart HPA kém hơn ở Underprovision và Overutilization một cách nhất quán, do cách chia quota tham lam.
3. ProSmart HPA dùng vượt tổng quota 16–23% (W8), nên ưu thế về Overutilization và latency không cùng mức tài nguyên.
4. Phát hiện code ARM làm mất 25–37% quota (W9); sửa lỗi này đưa Underprovision ở 5R-50% về gần 0 như bài báo công bố.
5. Chỉ số CPU của bài báo không phản ánh đầy đủ trải nghiệm người dùng (C1); chia quota theo ưu tiên (C3) chỉ cải thiện nhỏ ở Overutilization.
6. Với tải đột biến (C2), Smart HPA vẫn tốt hơn về tài nguyên nhưng không cải thiện latency và bị flapping (đảo chiều scale khoảng 43 lần mỗi lần chạy so với 0 của K8s HPA).

Hướng phát triển: thêm cửa sổ ổn định khi scale down cho Smart HPA rồi chạy lại tải đột biến để kiểm tra flapping và request lỗi; cho phép service ưu tiên cao thu hồi quota đã cho mượn; sửa Algorithm 4 để ProSmart không vượt quota rồi so sánh lại trong cùng mức tài nguyên; chạy thêm kịch bản và tăng số lần chạy lên 10 như bài báo.

Tài liệu tham khảo

[1] H. Ahmad, C. Treude, M. Wagner, C. Szabo, "Towards resource-efficient reactive and proactive auto-scaling for microservice architectures," Journal of Systems and Software, vol. 225, 112390, 2025. doi:10.1016/j.jss.2025.112390

[2] H. Ahmad, C. Treude, M. Wagner, C. Szabo, "Smart HPA: A resource-efficient horizontal pod auto-scaler for microservice architectures," IEEE ICSA 2024. doi:10.1109/ICSA59870.2024.00013

[3] Replication package Smart HPA: https://doi.org/10.6084/m9.figshare.25393408

[4] Replication package ProSmart HPA: https://github.com/HussainAhmad05/ProSmart_HPA

[5] Online appendix (hình chi tiết): https://doi.org/10.6084/m9.figshare.28028417

[6] Google, Online Boutique: https://github.com/GoogleCloudPlatform/microservices-demo

[7] k3s documentation: https://docs.k3s.io

[8] Locust documentation: https://docs.locust.io

[9] Kubernetes, Horizontal Pod Autoscaling: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[10] S. J. Taylor, B. Letham, "Forecasting at scale," The American Statistician, vol. 72, no. 1, pp. 37–45, 2018.

Phụ lục A. Mã nguồn và dữ liệu

Toàn bộ mã nguồn, script và dữ liệu thô nằm trong repo của nhóm. Các file chính:

File / thư mục	Chức năng
`smarthpa/run_experiment.ps1`	Một lần chạy: reset, cooldown, chạy auto-scaler + recorder + Locust, kiểm tra, báo cáo
`smarthpa/run_batch.ps1`	Chạy theo cặp xen kẽ, bỏ qua lần đã có, tự chạy lại tối đa 3 lần
`smarthpa/k8shpa_observer.py`	Ghi Knowledge Base cho K8s HPA cùng định dạng Smart HPA
`smarthpa/cluster_recorder.py`	Ghi replica, CPU, pod Pending/Unschedulable mỗi chu kỳ
`smarthpa/check_run.py`	Loại lần chạy mất kết nối
`smarthpa/make_comparison.py`	Bảng nhất quán, TB ± ĐLC, Mann-Whitney U, biểu đồ
`smarthpa/make_user_metrics.py`	Chỉ số phía người dùng (C1)
`smarthpa/Priority_Resource_Manager.py`	ARM theo độ ưu tiên (C3) và biến thể đối chứng
`smarthpa/locustfile_spike.py`, `make_flapping.py`	Tải đột biến và chỉ số ổn định (C2)
`prosmarthpa/`	Controller ProSmart (Algorithm 3–4), script train PROPHET và script so sánh với model của tác giả (Bảng 5.4, Hình 5.6)
`results/<kịch bản>/<auto-scaler>/run<N>/`	Dữ liệu thô từng lần chạy